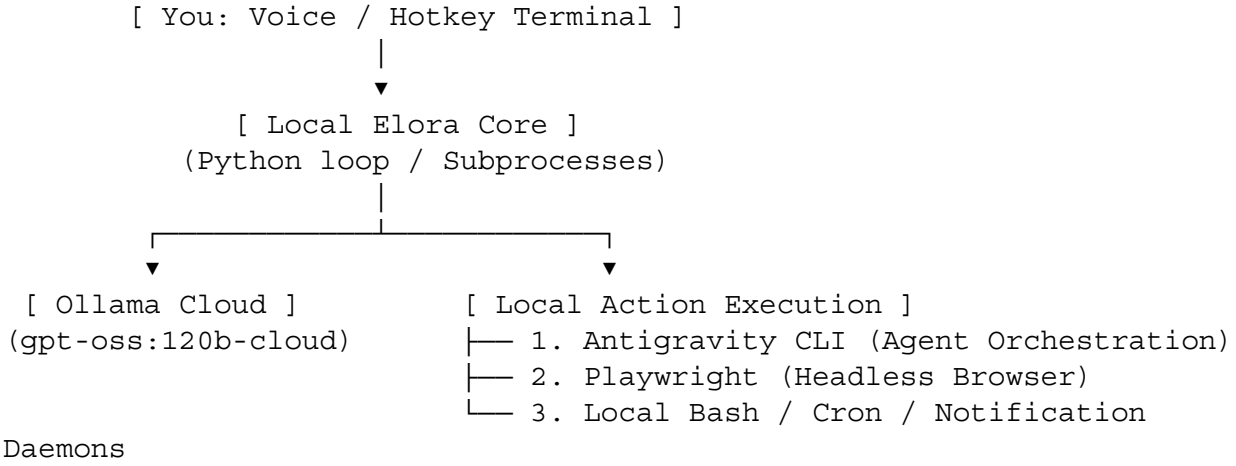
1. Architecture: The Elora Design Blueprint

To ensure Elora doesn't lag or freeze your laptop, she should be written as a lightweight local Python loop using uv for ultra-fast execution, communicating with a massive cloud reasoning model via Ollama.



The Workflow Strategy

1. **Low-Resource Brain:** You pipe your command into Elora. Elora sends the text payload to gpt-oss:120b-cloud (or similar heavyweight models) via your local signed-in Ollama client (ollama signin).
2. **Task Delegation:** The cloud model determines what tool is required and returns a structured JSON instruction (Tool Call).
3. **Local Action:** The local Python script handles the instruction, spawns a background thread or a quick tool instance, and frees your terminal instantly.

2. Improving the Core Features

Feature A: Prompting Other Agents (Antigravity CLI / Specialized Tools)

Instead of Elora trying to solve complex coding tasks or massive data calculations herself, she should hand them off to dedicated sub-agents that spin up, finish the job, and close down.

- **How it improves:** If you tell Elora, *"Write a complete automation backend for my game project,"* she won't try to draft 500 lines of code in a single prompt. Instead, she writes a configuration payload and passes it directly to **Antigravity CLI** or **Claude Code via Ollama** in a background tmux window.
- **User Experience:** Elora will send a desktop notification: *"Task handed over to Antigravity CLI in background session 'elora-dev'. I'll alert you when it's compiled."*

Feature B: Personalized News Aggregator & Intelligent Navigation

Having an AI open a heavy graphical browser like Chrome to scroll through news is a major resource drain. We can optimize this completely.

- **The News Engine (Backend):** Elora can run a quick, silent Python task using playwright or standard curl requests to fetch RSS feeds and tech blogs specified by you.
- **The Delivery (Frontend):**
    - **The "Skim" Option:** Elora summarizes the top 5 articles into a clean Markdown format and displays them inside a floating terminal or widget.
    - **The "Deep Dive" Option:** If you say *"Open the full article for number 2,"* Elora uses a bash command to launch your preferred lightweight browser (like Chromium or a specialized profile) and loads the exact URL instantly.

3. Implementation Plan (Getting Started)

You can structure Elora's local directory efficiently using uv:

```
uv init elora
cd elora
uv add ollama playwright
```

The Core Orchestration Script (main.py)

Here is a conceptual structure of how Elora's decision loop routes your tasks locally while keeping your system responsive:

```
import os
import subprocess
from ollama import Client

# Initialize the Ollama client pointing to the Cloud routing
client = Client()

def ask_elora(user_prompt):
    # System instructions guiding Elora to choose local tools
    system_instruction = (
        "You are Elora, an OS orchestrator. You route tasks to local
Linux tools. "
        "Respond strictly with a JSON action block. Valid tools:
'antigravity', 'browser', 'news_fetch'."
    )

    response = client.chat(
        model='gpt-oss:120b-cloud', # Offloads heavy lifting
completely
        messages=[
            {'role': 'system', 'content': system_instruction},
            {'role': 'user', 'content': user_prompt}
        ]
    )
```

```
    return response['message']['content']

def execute_action(action_json):
    # Pseudo-logic to parse Elora's intent
    # Example 1: Triggering another CLI agent in the background
    if "antigravity" in action_json:
        subprocess.Popen(["tmux", "new-session", "-d", "-s",
"elora-agent", "antigravity-cli run --task ..."])
        subprocess.run(["notify-send", "Elora", "Spawned Antigravity
agent in background."])

    # Example 2: Prompting web navigation
    elif "browser" in action_json:
        url = "https://news.ycombinator.com" # Parsed target URL
        subprocess.Popen(["xdg-open", url]) # Launches your default
system browser instantly

# Example Run
# command = "Fetch my tech news and open the browser to hackernews"
# res = ask_elora(command)
# execute_action(res)
```

4. Elevating the "Jarvis" Vibe on Linux

To make Elora feel like an actual entity rather than just a script you run, tie her into your desktop's ecosystem:

- **Trigger Hotkey:** Bind a key combination in your window manager configurations to launch a small terminal prompt (e.g., alacritty --class EloraPrompt -e python main.py).
- **Desktop Notifications:** Use notify-send in your scripts so Elora can quietly pop up notifications in the corner of your screen when background tasks complete.
- **Audio cues:** If you want sound, keep ultra-light audio files (.wav) locally and use aplay or mpv to play a subtle chime when she finishes fetching your data.